Лабораторна робота 3

Розроблення моделі Transformer для аналізу текстових та часових послідовностей

У лабораторній роботі розглядається архітектура Transformer, яка базується на механізмі самоуваги (Self-Attention) і застосовується для оброблення послідовних даних. Реалізуються механізми самоуваги та багатоголової уваги, будується модель Transformer для класифікації текстів або прогнозування часових рядів технологічних даних. Виконується візуалізація ваг уваги та аналіз їхньої інтерпретованості.

Мета роботи

Вивчити принципи побудови та функціонування нейронних мереж архітектури Transformer для оброблення послідовних даних. Навчитися реалізовувати механізми масштабованої скалярно-добуткової самоуваги (Scaled Dot-Product Attention) та багатоголової уваги (Multi-Head Attention), застосовувати позиційне кодування і будувати модель Transformer Encoder для класифікації текстів або прогнозування часових рядів. Дослідити вплив параметрів архітектури на якість моделювання, виконати візуалізацію ваг уваги та проаналізувати можливості їх інтерпретації.

Теоретичний матеріал

1. Архітектура Transformer

Transformer – архітектура нейронної мережі, запропонована у праці *Attention Is All You Need* (Vaswani et al., 2017). Її ключовою особливістю є використання механізму уваги для моделювання залежностей між елементами послідовності.

Початкова архітектура Transformer складається з двох основних компонентів: енкодера (Encoder), який формує представлення вхідної послідовності, та декодера (Decoder), який генерує вихідну послідовність.

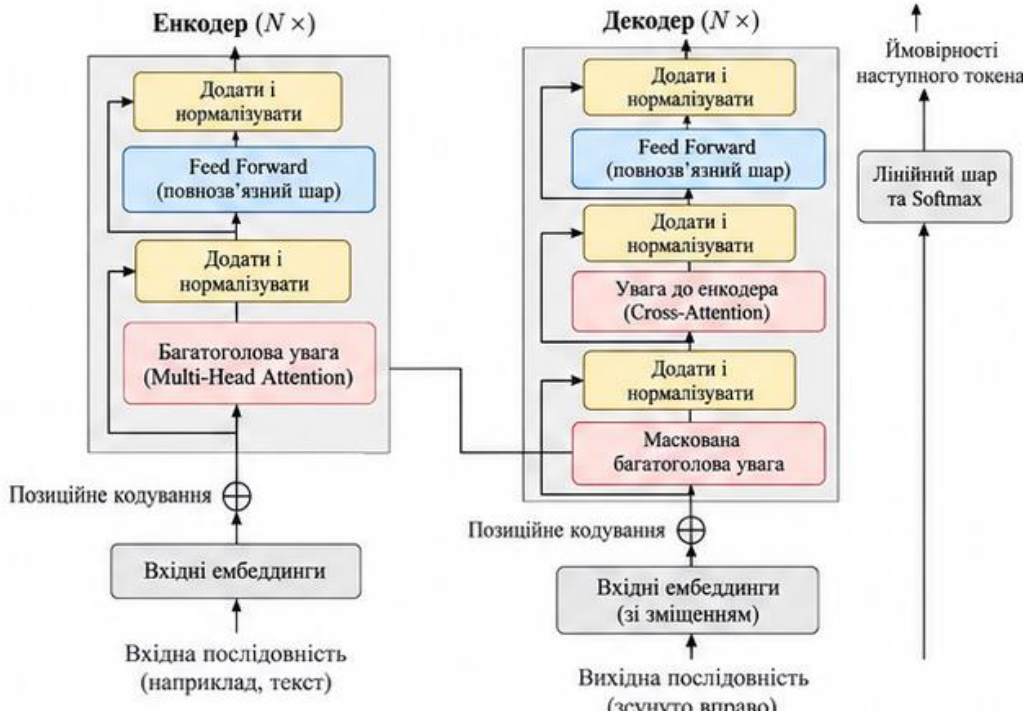


Рис. 1. Загальна архітектура моделі Transformer (енкодер–декодер).

Проте для задач класифікації текстів і прямого прогнозування часових рядів часто достатньо архітектури на основі **Transformer Encoder**, без декодера.

Кожен блок енкодера містить механізм багатоголової самоуваги та позиційно-незалежну повнозв'язну мережу (Feed-Forward Network, FFN). Також використовуються залишкові з'єднання (Residual Connections) і нормалізація (Layer Normalization).

2. Векторне представлення послідовностей

Механізм уваги Transformer виконує операції над числовими векторами однакової розмірності. Тому перед подачею даних у шари Transformer необхідно сформувати відповідне векторне представлення кожного елемента послідовності.

Для текстових даних використовується шар Embedding, який перетворює індекс кожного токена на вектор розмірності d_{model}:

$$e_t = \text{Embedding}(w_t),$$

де w_t – індекс токена в словнику, $e_t \in \mathbb{R}^{d_{\text{model}}}$ – його векторне представлення.

Якщо довжина тексту становить $T = 100$ токенів, а розмірність ембеддингу $d_{\text{model}} = 64$, то після шару Embedding один текст буде представлений матрицею

$$E \in \mathbb{R}^{100\times 64}.$$

Для часових рядів використовується лінійна проєкція вектора вимірювань:

$$e_t = W_e x_t + b_e,$$

де $W_e \in \mathbb{R}^{d_{\text{model}}\times F}$ – навчувана матриця ваг, b_e – вектор зміщення.

Наприклад, якщо технологічна система має 4 вимірювані параметри, а модель використовує $d_{\text{model}} = 64$, то вхідний вектор із 4 значень перетворюється на вектор із 64 компонентів.

В обох випадках результат має однакову структуру:

$$E \in \mathbb{R}^{T\times d_{\text{model}}}.$$

Завдяки цьому наступні шари Transformer можуть працювати як із текстовими, так і з числовими послідовностями.

3. Позиційне кодування (Positional Encoding)

Для врахування позиції кожного елемента до його векторного представлення додається спеціальний позиційний вектор.

Один із класичних способів – синусоїдальне позиційне кодування:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right),$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right),$$

де pos – позиція елемента в послідовності, i – індекс пари компонент позиційного вектора.

Позиційний вектор додається до ембеддингу:

$$z_t = e_t + PE_t.$$

У результаті нейронна мережа отримує інформацію не лише про характеристики елемента, а й про його розташування в послідовності.

Крім синусоїдального позиційного кодування, використовують навчувані позиційні ембеддинги (Learned Positional Embeddings). Вплив цих двох підходів можна дослідити в індивідуальних завданнях.

4. Механізм самоуваги (Self-Attention)

Механізм самоуваги (Self-Attention) є основним компонентом архітектури Transformer. Він дозволяє кожному елементу послідовності враховувати інформацію про інші елементи цієї самої послідовності та визначати, які з них є найбільш важливими для формування його нового представлення.

4.1. Формування матриць Query, Key та Value

Нехай після векторного представлення та позиційного кодування отримано матрицю

$$Z \in \mathbb{R}^{T \times d_{\text{model}}},$$

де T – довжина послідовності, d_{model} – розмірність представлення кожного елемента.

Для обчислення самоуваги формують три матриці:

- **Query (Q)** – матриця запитів, яка визначає, яку інформацію шукає кожен елемент послідовності.
- **Key (K)** – матриця ключів, за допомогою яких оцінюють відповідність елементів запитам.
- **Value (V)** – матриця значень, які використовуються для формування нових представлень елементів.

Матриці обчислюють за допомогою трьох незалежних лінійних перетворень:

$$Q = ZW^Q, \qquad K = ZW^K, \qquad V = ZW^V,$$

де W^Q, W^K, W^V – матриці ваг, які визначаються під час навчання нейронної мережі.

Розмірності матриць:

$$W^Q, W^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \qquad W^V \in \mathbb{R}^{d_{\text{model}} \times d_v}.$$

Відповідно:

$$Q, K \in \mathbb{R}^{T \times d_k}, \qquad V \in \mathbb{R}^{T \times d_v}.$$

Таким чином, кожен елемент послідовності отримує три векторні представлення, які використовуються для встановлення зв’язків між елементами.

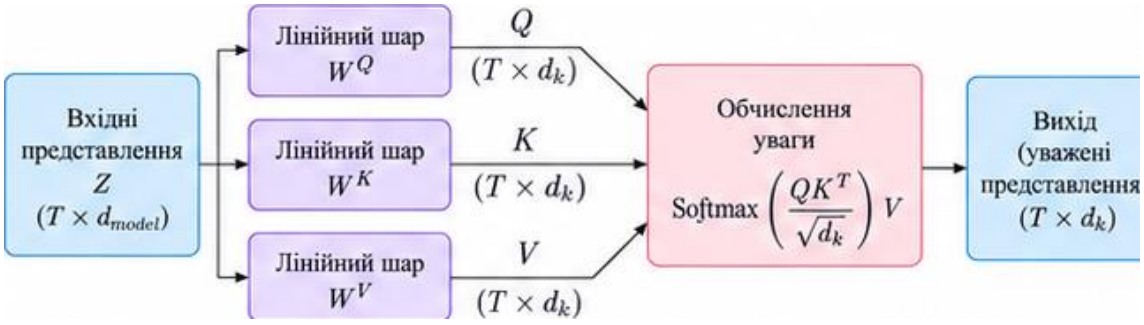


Схема обчислення механізму самоуваги

Рис. 2. Схема обчислення механізму самоуваги.

4.2. Обчислення оцінок подібності

Щоб визначити, наскільки один елемент послідовності пов’язаний з іншим, обчислюють скалярний добуток відповідних векторів Query та Key.

Для елементів із позиціями i та j:

$$s_{ij} = \frac{q_i k_j^\top}{\sqrt{d_k}},$$

де q_i – вектор запиту для елемента i, k_j – вектор ключа елемента j, d_k – розмірність векторів запитів і ключів.

Для всієї послідовності оцінки подібності формують матрицю

$$S = \frac{QK^\top}{\sqrt{d_k}}. \quad S \in \mathbb{R}^{T\times T}.$$

Кожен її рядок характеризує зв'язок певного елемента з усіма елементами послідовності.

Масштабування .Якщо розмірність векторів є великою,то ділення на $\sqrt{d_k}$ допомагає стабілізувати величину оцінок подібності.Саме тому механізм називається *Scaled Dot-Product Attention* – масштабована скалярно-добуткова увага.

4.3. Нормалізація ваг уваги

Отримані оцінки подібності перетворюють на ваги за допомогою функції Softmax, яка застосовується до кожного рядка матриці окремо:

$$a_{ij} = \frac{\exp(s_{ij})}{\sum_{m=1}^{T} \exp(s_{im})}.$$

Кожна вага має значення в інтервалі від 0 до 1:

$$0 \le a_{ij} \le 1,$$

а сума ваг у кожному рядку дорівнює одиниці:

$$\sum_{j=1}^{T} a_{ij} = 1.$$

Вага a_{ij} визначає, з яким коефіцієнтом значення елемента j враховується під час формування нового представлення елемента i.

4.4. Формування вихідного представлення

Після обчислення ваг уваги кожен вихідний вектор формується як зважена сума векторів:

$$o_i = \sum_{j=1}^{T} a_{ij}\, v_j.$$

У матричному вигляді:

$$O = AV.$$

Отже, повна формула механізму самоуваги має вигляд:

$$\boxed{\text{Attention}(Q,K,V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V} \qquad O \in \mathbb{R}^{T\times d_v}.$$

Кожен вихідний вектор містить інформацію, сформовану з урахуванням усієї доступної послідовності.

5. Багатоголова увага (Multi-Head Attention)

Використання одного механізму уваги дозволяє встановлювати залежності між елементами послідовності лише в одному навченому просторі представлень. Однак дані можуть містити декілька різних типів взаємозв'язків.

Для цього в Transformer використовується багатоголова увага (Multi-Head Attention, MHA), у якій декілька механізмів уваги обчислюються паралельно.

5.1. Принцип роботи багатоголової уваги

Нехай кількість голів уваги дорівнює H. Для кожної голови використовуються власні матриці лінійних перетворень:

$$Q_i = ZW_i^Q, \quad K_i = ZW_i^K, \quad V_i = ZW_i^V, \text{ де } i = 1,2,\dots,H.$$

Кожна голова незалежно обчислює результат самоуваги:

$$head_i = \text{Attention}(Q_i, K_i, V_i).$$

Отримані виходи об'єднуються за допомогою операції конкатенації:

$$O_{\text{concat}} = \text{Concat}(head_1, \dots, head_H).$$

Після цього застосовується вихідне лінійне перетворення:

$$\boxed{\text{MultiHead}(Z) = \text{Concat}(head_1, \dots, head_H)W^O}$$

де W^O – навчувана матриця, яка перетворює об'єднані результати у вихідний простір представлень.

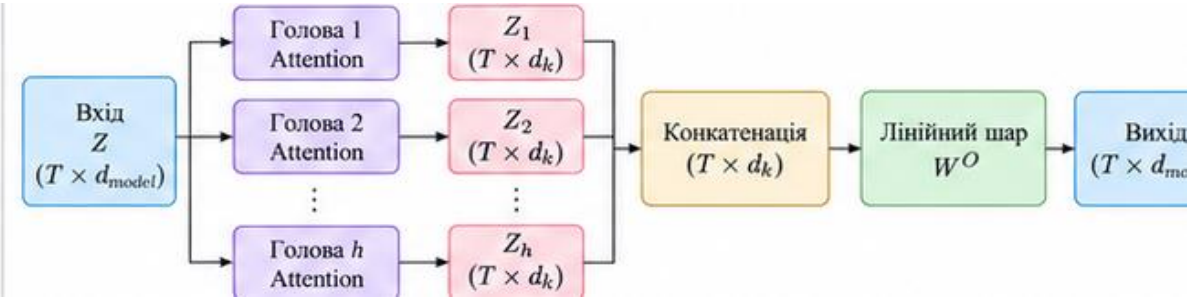


Рис. 3. Схема багатоголової уваги.

5.2. Розмірності тензорів

У типовій реалізації розмірність представлення рівномірно розподіляється між головами:

$$d_k = d_v = \frac{d_{\text{model}}}{H}.$$

Наприклад, для $d_{\text{model}} = 64$ і $H = 4$ кожна голова опрацьовує вектори розмірності 16.

5.3. Аналіз різних голів уваги

Кожна голова формує власну матрицю ваг:

$$A^{(i)} = \text{Softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}}\right).$$

Різні голови можуть навчитися виділяти різні залежності між елементами. Водночас ці закономірності не задаються заздалегідь: вони виникають у процесі навчання, і не кожна голова обов'язково набуває чітко відмінної функції.

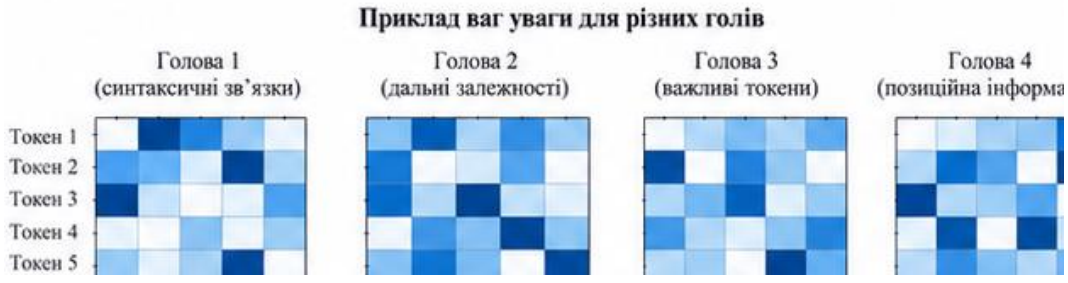


Рис. 4. Приклад ваг уваги для різних голів багатоголової уваги.

У лабораторній роботі необхідно буде отримати фактичні матриці ваг навченої моделі та порівняти їх між різними головами.

6. Структура шару Transformer Encoder

Transformer Encoder складається з декількох однакових за структурою шарів, які послідовно перетворюють вхідне представлення.

Кожен шар містить два основні блоки:

1. Multi-Head Self-Attention – формує представлення з урахуванням взаємозв'язків між елементами послідовності.

2. Feed-Forward Network – виконує нелінійне перетворення представлення кожного елемента.

Для стабілізації навчання застосовуються залишкові з'єднання та нормалізація.

6.1. Повнозв'язна мережа (Feed-Forward Network)

Після механізму багатоголової уваги кожен елемент послідовності проходить через двошарову повнозв'язну мережу:

$$FFN(z) = W_2\,\sigma(W_1 z + b_1) + b_2,$$

де W_1, W_2 – матриці ваг, b_1, b_2 – вектори зміщення, σ – нелінійна функція активації.

На відміну від самоуваги, FFN не виконує безпосереднього обміну інформацією між різними позиціями послідовності. Однакова за параметрами мережа застосовується незалежно до кожного позиційного представлення.

6.2. Нормалізація шарів (Layer Normalization)

Layer Normalization нормалізує компоненти вектора представлення в межах окремого елемента послідовності:

$$\mathrm{LN}(x) = \gamma\,\frac{x - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta,$$

де μ та σ^2 – середнє значення та дисперсія компонент вектора, γ, β – навчувані параметри, ε – мала додатна константа.

Існують два поширені варіанти розташування нормалізації: Post-LN, коли нормалізація застосовується після додавання залишкового з'єднання, та Pre-LN, коли вона виконується перед основним перетворенням.Для практичної реалізації доцільно використовувати Pre-LN, особливо для полегшення стабільного навчання.

Приклад. Реалізація моделі Transformer засобами TensorFlow/Keras

Частина 1. Реалізація механізмів Self-Attention та Multi-Head Attention

Завдання. Реалізувати механізм масштабованої скалярно-добуткової самоуваги (Scaled Dot-Product Attention) за допомогою TensorFlow, виконати обчислення ваг уваги для невеликої послідовності та побудувати їхню теплову карту. Реалізувати багатоголову увагу (Multi-Head Attention), перевірити розмірності тензорів, дослідити результати роботи окремих голів і побудувати блок Transformer Encoder.

Базовий приклад складається з таких етапів: формування тестової послідовності, обчислення Self-Attention, аналіз матриці ваг, реалізація Multi-Head Attention, дослідження окремих голів і створення шару Transformer Encoder. Після цього реалізовані компоненти будуть використані для класифікації текстів і прогнозування часових рядів.

1. Підключення бібліотек

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

from tensorflow import keras
from tensorflow.keras import layers

np.random.seed(42)
tf.random.set_seed(42)

print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
```

Для виконання прикладу достатньо TensorFlow 2.x із підтримкою tf.keras. Його можна запускати в Google Colab або локальному Python-середовищі.

2. Формування тестової послідовності

Для демонстрації механізму самоуваги використаємо послідовність із трьох елементів, кожен із яких представлений двовимірним вектором:

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

У цьому прикладі довжина послідовності $T = 3$, розмірність представлення $d_{\text{model}} = 2$, кількість послідовностей у пакеті $B = 1$.

```
X = tf.constant(
    [
        [1.0, 0.0],
        [0.0, 1.0],
        [1.0, 1.0]
    ],
    dtype=tf.float32
)

X = tf.expand_dims(X, axis=0)

print("Shape:", X.shape)
print(X.numpy())
```

Розмірність тензора:

```
Shape: (1, 3, 2)
```

Перша розмірність відповідає кількості послідовностей у пакеті, друга – довжині послідовності, третя – кількості ознак кожного елемента.

3. Реалізація Scaled Dot-Product Attention

Основна формула механізму уваги:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)V.$$

Для її реалізації необхідно виконати матричне множення Query та Key, масштабування, нормалізацію за допомогою Softmax і формування зваженої суми Value.

```
def scaled_dot_product_attention(q, k, v, mask=None):

    # Скалярні добутки Query та Key
    scores = tf.matmul(q, k, transpose_b=True)

    # Масштабування
    dk = tf.cast(tf.shape(k)[-1], scores.dtype)
    scores = scores / tf.math.sqrt(dk)
```

```
    # Необов'язкове маскування
    # 1 / True – дозволена позиція
    # 0 / False – заборонена позиція
    if mask is not None:
        mask = tf.cast(mask, tf.bool)
        scores = tf.where(
            mask,
            scores,
            tf.cast(-1e9, scores.dtype)
        )

    # Нормалізація ваг
    attention_weights = tf.nn.softmax(
        scores, axis=-1
    )

    # Зважена сума Value
    output = tf.matmul(attention_weights, v)

    return output, attention_weights
```

4. Перевірка механізму самоуваги

Для спрощення перевірки використаємо вихідну послідовність одночасно як Query, Key та Value:

$$Q = K = V = X.$$

Це дозволить порівняти програмні результати з ручним обчисленням матриці уваги.

```
output, weights = scaled_dot_product_attention(
    Q, K, V
)

print("Ваги уваги:")
print(np.round(weights.numpy()[0], 3))

print("\nРезультат Self-Attention:")
print(np.round(output.numpy()[0], 3))
```

Перевірка нормалізації

Для контролю коректності реалізації необхідно перевірити, що сума ваг уваги для кожного запиту дорівнює одиниці:

```
row_sums = tf.reduce_sum(weights, axis=-1)

print("Суми ваг уваги:")
print(row_sums.numpy())

tf.debugging.assert_near(
    row_sums,
    tf.ones_like(row_sums)
)
```

5. Візуалізація матриці уваги

Для візуального аналізу результатів побудуємо теплову карту матриці уваги. Колір кожної клітинки відображатиме значення відповідної ваги.

```
attention_matrix = weights.numpy()[0]

plt.figure(figsize=(6, 5))

plt.imshow(
    attention_matrix,
    cmap="Blues",
    vmin=0,
    vmax=1
```

```
)

plt.colorbar(label="Вага уваги")

plt.xticks(
    range(3),
    ["Елемент 1", "Елемент 2", "Елемент 3"]
)

plt.yticks(
    range(3),
    ["Елемент 1", "Елемент 2", "Елемент 3"]
)

for i in range(3):
    for j in range(3):
        plt.text(
            j, i,
            f"{attention_matrix[i, j]:.3f}",
            ha="center",
            va="center",
            color="black"
        )

plt.xlabel("Елемент, на який спрямована увага")
plt.ylabel("Елемент, що формує запит")
plt.title("Матриця Self-Attention")

plt.tight_layout()
plt.show()
```

Важливо, що в цьому прикладі матриці Query, Key та Value задані без навчання. Отримані ваги демонструють математичний принцип роботи самоуваги, але ще не відображають закономірностей, виявлених навченою нейронною мережею.

6. Реалізація багатоголової уваги (Multi-Head Attention)

На відміну від звичайної самоуваги, багатоголова увага використовує декілька незалежних наборів матриць Query, Key та Value. Це дозволяє формувати кілька представлень взаємозв'язків між елементами послідовності.

Для кожної голови уваги обчислюється:

$$head_i = \text{Attention}(Q_i, K_i, V_i).$$

Після цього результати конкатенуються та перетворюються за допомогою навчуваного лінійного шару:

$$\text{MHA}(X) = \text{Concat}(head_1, \dots, head_H)W^O.$$

Для практичної реалізації використаємо такі параметри:

- розмірність представлення d_model = 64;
- кількість голів num_heads = 4;
- розмірність однієї голови head_dim = 16.

6.1. Власна реалізація Multi-Head Attention

Щоб зрозуміти внутрішню структуру механізму, реалізуємо власний клас на основі tf.keras.layers.Layer та раніше створеної функції scaled_dot_product_attention().

```
class MultiHeadSelfAttention(layers.Layer):

    def __init__(self, d_model=64, num_heads=4):
        super().__init__()
```

```
        if d_model % num_heads != 0:
            raise ValueError(
                "d_model має ділитися на num_heads"
            )

        self.d_model = d_model
        self.num_heads = num_heads
        self.depth = d_model // num_heads

        # Навчувані проєкції Q, K, V
        self.Wq = layers.Dense(d_model)
        self.Wk = layers.Dense(d_model)
        self.Wv = layers.Dense(d_model)

        # Вихідна проєкція
        self.Wo = layers.Dense(d_model)

    def split_heads(self, x):

        batch_size = tf.shape(x)[0]
        seq_len = tf.shape(x)[1]

        x = tf.reshape(
            x,
            [
                batch_size,
                seq_len,
                self.num_heads,
                self.depth
            ]
        )

        return tf.transpose(x, perm=[0, 2, 1, 3])

    def call(self, x, mask=None):

        # Формування Query, Key та Value
        q = self.Wq(x)
        k = self.Wk(x)
        v = self.Wv(x)

        # Розподіл на голови
        q = self.split_heads(q)
        k = self.split_heads(k)
        v = self.split_heads(v)

        # Самоувага для кожної голови
        attention_output, attention_weights = (
            scaled_dot_product_attention(q, k, v, mask)
        )

        # Повернення до початкового порядку осей
        attention_output = tf.transpose(
            attention_output,
            perm=[0, 2, 1, 3]
        )

        batch_size = tf.shape(x)[0]
        seq_len = tf.shape(x)[1]

        # Об'єднання голів
        concat = tf.reshape(
            attention_output,
            [batch_size, seq_len, self.d_model]
        )

        # Вихідне лінійне перетворення
        output = self.Wo(concat)
```

```
        return output, attention_weights
```

6.2. Перевірка розмірностей тензорів

Сформуємо дві випадкові послідовності довжини 8 із розмірністю представлення 64.

```
X_test = tf.random.normal((2, 8, 64))

mha_custom = MultiHeadSelfAttention(
    d_model=64,
    num_heads=4
)

output_mha, attention_scores = mha_custom(X_test)

print("Вхід:", X_test.shape)
print("Вихід:", output_mha.shape)
print("Матриці уваги:", attention_scores.shape)
```

Тензор ваг має чотири виміри:

$$A \in \mathbb{R}^{B \times H \times T \times T}.$$

У наведеному прикладі це означає, що для кожної з двох послідовностей формуються чотири окремі матриці уваги розміром 8×8.

Перевіримо нормалізацію:

```
row_sums = tf.reduce_sum(
    attention_scores,
    axis=-1
)

tf.debugging.assert_near(
    row_sums,
    tf.ones_like(row_sums),
    atol=1e-5
)

print("Перевірку нормалізації пройдено")
```

Це підтверджує, що сума ваг кожного рядка матриці уваги приблизно дорівнює одиниці з урахуванням числової точності обчислень.

7. Використання готового шару MultiHeadAttention у Keras

У TensorFlow/Keras механізм багатоголової уваги реалізований у шарі tf.keras.layers.MultiHeadAttention.

Він автоматично формує проєкції Query, Key та Value, розділяє представлення на голови, обчислює ваги уваги й об’єднує результати.

```
mha = layers.MultiHeadAttention(
    num_heads=4,
    key_dim=16,
    dropout=0.0
)

output, scores = mha(
    query=X_test,
    value=X_test,
    key=X_test,
    return_attention_scores=True
)

print("Вихід MHA:", output.shape)
print("Ваги уваги:", scores.shape)
```

Очікуваний результат:

```
Вихід MHA: (2, 8, 64)
Ваги уваги: (2, 4, 8, 8)
```

Параметр num_heads=4 задає кількість голів, а key_dim=16 – розмірність простору запитів і ключів кожної голови.Параметр return_attention_scores=True дозволяє отримати ваги уваги для подальшого аналізу.

8. Візуалізація ваг окремих голів

Для дослідження багатоголової уваги візуалізуємо матрицю кожної голови окремо.

Використаємо ваги, отримані з шару MultiHeadAttention.

```
# Перша послідовність із пакета
attention = scores.numpy()[0]

# Чотири голови уваги
for head in range(4):

    plt.figure(figsize=(6, 5))

    plt.imshow(
        attention[head],
        cmap="Blues",
        vmin=0,
        vmax=1
    )

    plt.colorbar(label="Вага уваги")

    plt.xlabel("Позиція Key")
    plt.ylabel("Позиція Query")

    plt.title(f"Голова уваги {head + 1}")

    plt.tight_layout()
    plt.show()
```

Кожен графік відображає власну матрицю уваги розміром 8×8.

Під час аналізу необхідно звернути увагу на такі особливості:

- Чи мають матриці різних голів однакову структуру?
- Чи зосереджена увага на головній діагоналі, що відповідає зверненню елементів до самих себе?
- Чи є позиції, яким багато інших елементів надають підвищену вагу?
- Наскільки рівномірно розподіляється увага між усіма елементами?

На цьому етапі шар ще не навчений на конкретному завданні, тому візуалізація демонструє переважно результат випадкової ініціалізації. Змістовний аналіз закономірностей уваги потрібно виконувати після навчання Transformer на текстових або часових даних.

9. Побудова шару Transformer Encoder

Наступним етапом є об’єднання механізму Multi-Head Attention із повнозв’язною мережею, залишковими з’єднаннями та нормалізацією.

Реалізуємо шар енкодера із нормалізацією Pre-LN:

$$U = X + \mathrm{MHA}\big(\mathrm{LN}(X)\big),$$

$$Y = U + \mathrm{FFN}\big(\mathrm{LN}(U)\big).$$

Реалізація Transformer Encoder Block

```
class TransformerEncoderBlock(layers.Layer):
```

```
def __init__(
    self,
    d_model=64,
    num_heads=4,
    ff_dim=128,
    dropout=0.1
):
    super().__init__()

    # Багатоголова увага
    self.attention = layers.MultiHeadAttention(
        num_heads=num_heads,
        key_dim=d_model // num_heads,
        dropout=dropout
    )

    # Повнозв'язна мережа
    self.ffn = keras.Sequential([
        layers.Dense(ff_dim, activation="relu"),
        layers.Dense(d_model)
    ])

    # Нормалізація
    self.norm1 = layers.LayerNormalization(
        epsilon=1e-6
    )

    self.norm2 = layers.LayerNormalization(
        epsilon=1e-6
    )

    # Регуляризація
    self.dropout1 = layers.Dropout(dropout)
    self.dropout2 = layers.Dropout(dropout)

def call(
    self,
    x,
    training=False,
    attention_mask=None,
    return_attention_scores=False
):

    # Pre-LN перед увагою
    z = self.norm1(x)

    attn_output, attn_scores = self.attention(
        query=z,
        value=z,
        key=z,
        attention_mask=attention_mask,
        return_attention_scores=True,
        training=training
    )

    # Перше залишкове з'єднання
    x = x + self.dropout1(
        attn_output,
        training=training
    )

    # Pre-LN перед FFN
    z = self.norm2(x)

    ffn_output = self.ffn(z, training=training)

    # Друге залишкове з'єднання
    x = x + self.dropout2(
        ffn_output,
```

```
            training=training
        )

        if return_attention_scores:
            return x, attn_scores

        return x
```

У реалізованому шарі багатоголова увага формує контекстні представлення елементів, а Feed-Forward Network виконує їхнє нелінійне перетворення.

Окрім вихідного тензора, реалізація дозволяє за необхідності повертати матриці уваги. Це важливо для подальшої візуалізації та аналізу інтерпретованості моделі.

Перевірка Encoder Block

```
encoder = TransformerEncoderBlock(
    d_model=64,
    num_heads=4,
    ff_dim=128,
    dropout=0.1
)

Y, A = encoder(
    X_test,
    training=False,
    return_attention_scores=True
)

print("Вхід:", X_test.shape)
print("Вихід:", Y.shape)
print("Ваги уваги:", A.shape)
```

Очікувані розмірності:

```
Вхід: (2, 8, 64)
Вихід: (2, 8, 64)
Ваги уваги: (2, 4, 8, 8)
```

Розмірність виходу енкодера збігається з розмірністю входу, що дозволяє послідовно об’єднувати декілька таких блоків для формування глибшої моделі Transformer.

Приклад. Класифікація текстових послідовностей за допомогою Transformer

Частина 2. Побудова та навчання Transformer для класифікації текстів

Завдання. Побудувати модель Transformer Encoder засобами TensorFlow/Keras для автоматичної класифікації текстових документів за тематичними категоріями. Виконати попередню обробку текстів, токенізацію, формування ембеддингів, позиційне кодування, навчання та тестування моделі. Оцінити якість класифікації за метриками Accuracy і F1-score, побудувати матрицю помилок, візуалізувати ваги багатоголової уваги та проаналізувати їхню структуру.

Для прикладу використовується набір даних **AG News**, що містить короткі англомовні новини чотирьох тематичних категорій: World, Sports, Business і Sci/Tech.

У TensorFlow Datasets цей набір доступний під назвою ag_news_subset. Він містить 120 000 навчальних та 7 600 тестових записів. Кожен запис складається з назви новини (title), її опису (description) та мітки класу (label).

1. Підключення бібліотек

Для реалізації використовуються TensorFlow/Keras, TensorFlow Datasets, NumPy, Matplotlib та scikit-learn.

У Google Colab за необхідності встановити відсутні бібліотеки:

```
%pip install -q tensorflow-datasets scikit-learn
```

Підключення:

```
import numpy as np
import tensorflow as tf
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt

from tensorflow import keras
from tensorflow.keras import layers

from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)

np.random.seed(42)
tf.random.set_seed(42)

print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
```

Для побудови моделі також використовується клас TransformerEncoderBlock, реалізований у попередній частині лабораторної роботи. Його код необхідно виконати перед створенням класифікатора.

2. Завантаження набору даних AG News

Набір даних завантажується безпосередньо з TensorFlow Datasets.

```
(train_full, test_full), info = tfds.load(
    "ag_news_subset",
    split=["train", "test"],
    with_info=True,
    shuffle_files=False
)

class_names = info.features["label"].names

print("Класи:", class_names)
print("Навчальних записів:", info.splits["train"].num_examples)
print("Тестових записів:", info.splits["test"].num_examples)
```

Для класифікації використовуватимемо назву й опис новини, об’єднавши їх в один текст. Такий підхід дозволяє моделі враховувати інформацію з обох текстових полів.

```
def prepare_text(example):

    text = tf.strings.join(
        [
            example["title"],
            example["description"]
        ],
        separator=" "
    )

    label = tf.cast(example["label"], tf.int32)

    return text, label


train_full = train_full.map(prepare_text)
test_raw = test_full.map(prepare_text)
```

Для перевірки завантажених даних виведемо декілька прикладів.

```
for text, label in train_full.take(3):

    print("Текст:")
    print(text.numpy().decode("utf-8"))

    print("Клас:", class_names[int(label.numpy())])
    print("-" * 60)
```

Після виконання коду мають відображатися тексти новин із відповідними тематичними категоріями.

3. Формування навчальної, валідаційної та тестової вибірок

Для прискорення навчання на звичайному комп'ютері використаємо підмножину AG News.

Рекомендований початковий обсяг:

Вибірка	Кількість текстів	Призначення
Training	8 000	Навчання параметрів
Validation	2 000	Контроль перенавчання
Test	7 600	Остаточне оцінювання

Навчальні й валідаційні записи вибираються з початкової навчальної вибірки після перемішування. Офіційний тестовий набір залишається окремим.

```
SEED = 42

train_shuffled = train_full.shuffle(
    buffer_size=120000,
    seed=SEED,
    reshuffle_each_iteration=False
)

train_raw = train_shuffled.take(8000)

val_raw = (
    train_shuffled
    .skip(8000)
    .take(2000)
)
```

Використання фіксованого seed і вимкнення повторного перемішування дозволяє відтворити цей поділ. Через випадковий вибір кількість об'єктів окремих класів може дещо відрізнятися, тому їхній розподіл доцільно перевірити.

```
def count_classes(dataset, n_classes):
    counts = np.zeros(n_classes, dtype=int)

    for _, label in dataset:
        counts[int(label.numpy())] += 1

    return counts


print(
    "Training:",
    count_classes(train_raw, len(class_names))
)

print(
    "Validation:",
    count_classes(val_raw, len(class_names))
)
```

У разі істотного дисбалансу слід виконати стратифікований поділ, щоб забезпечити достатню кількість прикладів кожного класу.

4. Токенізація та векторизація текстів

Transformer не може безпосередньо опрацьовувати текстові рядки. Тому тексти необхідно перетворити на послідовності числових індексів токенів.

Для цього використаємо шар TextVectorization, який забезпечує нормалізацію текстів, токенізацію, побудову словника та перетворення токенів на числові індекси.

Встановимо такі параметри:

$$V_{\max} = 10000, \qquad L_{\max} = 64,$$

де $V_{\max}$ – максимальний розмір словника, $L_{\max}$ – максимальна довжина текстової послідовності.

```
MAX_VOCAB = 10000
MAX_LEN = 64
BATCH_SIZE = 32

vectorizer = layers.TextVectorization(
    max_tokens=MAX_VOCAB,
    standardize="lower_and_strip_punctuation",
    split="whitespace",
    output_mode="int",
    output_sequence_length=MAX_LEN
)
```

Словник формується за навчальною вибіркою. Валідаційні й тестові тексти не використовуються для його побудови.

```
text_ds = train_raw.map(
    lambda text, label: text
)

vectorizer.adapt(
    text_ds.batch(256)
)

vocab = vectorizer.get_vocabulary()

print("Розмір словника:", len(vocab))
print("Перші токени:", vocab[:20])
```

У режимі output_mode="int" індекс 0 резервується для доповнювальних позицій, а індекс 1 - для невідомих токенів. Довжина послідовності встановлюється через output_sequence_length.

Приклад токенізації

```
sample_text = tf.constant([
    "The company develops new artificial intelligence systems."
])

token_ids = vectorizer(sample_text)

print("Текст:", sample_text.numpy()[0])
print("Індекси:", token_ids.numpy()[0])
print("Розмірність:", token_ids.shape)
```

Якщо після токенізації текст має менше ніж 64 токени, решта позицій заповнюється нулями. Якщо токенів більше – послідовність обрізається до 64.

Для перевірки відповідності індексів токенам:

```
ids = token_ids.numpy()[0]

tokens = [
    vocab[int(i)]
    for i in ids
    if i != 0
]
```

```
print("Токени:", tokens)
```

5. Підготовка тензорів для навчання

Для ефективного навчання використаємо tf.data.Dataset, який дозволяє виконувати пакетування, перемішування та попередню підготовку даних.

```
AUTOTUNE = tf.data.AUTOTUNE

def create_dataset(dataset, training=False):

    if training:
        dataset = dataset.shuffle(
            2000,
            seed=SEED
        )

    dataset = dataset.batch(BATCH_SIZE)

    dataset = dataset.map(
        lambda text, label: (
            vectorizer(text),
            label
        ),
        num_parallel_calls=AUTOTUNE
    )

    dataset = dataset.prefetch(AUTOTUNE)

    return dataset
```

Формування вибірок:

```
train_ds = create_dataset(
    train_raw,
    training=True
)

val_ds = create_dataset(val_raw)

test_ds = create_dataset(test_raw)
```

Перевіримо структуру одного пакета.

```
for X_batch, y_batch in train_ds.take(1):

    print("Форма X:", X_batch.shape)
    print("Форма y:", y_batch.shape)
    print("Тип X:", X_batch.dtype)
```

Очікуваний результат:

```
Форма X: (32, 64)
Форма y: (32,)
Тип X: int64
```

Кожен пакет містить 32 текстові послідовності довжиною 64 токени та відповідні мітки класів.

6. Реалізація шару Embedding та позиційного кодування

Для подачі текстових послідовностей до Transformer необхідно перетворити індекси токенів на вектори розмірності d_{model} та додати інформацію про їхнє розташування в тексті.

У цьому прикладі використаємо позиційне кодування (Learned Positional Embedding). Позиційні вектори визначатимуться під час навчання моделі.

Нехай w_t – індекс токена на позиції t. Тоді його початкове представлення:

$$z_t = \text{Embedding}(w_t) + PE_t,$$

де PE_t – навчуваний вектор позиції t.

Реалізація

```
class TokenAndPositionEmbedding(layers.Layer):

    def __init__(self, max_len, vocab_size, d_model):
        super().__init__()

        self.token_embedding = layers.Embedding(
            input_dim=vocab_size,
            output_dim=d_model
        )

        self.position_embedding = layers.Embedding(
            input_dim=max_len,
            output_dim=d_model
        )

    def call(self, token_ids):

        seq_len = tf.shape(token_ids)[1]

        positions = tf.range(
            start=0,
            limit=seq_len,
            delta=1
        )

        token_vectors = self.token_embedding(token_ids)
        position_vectors = self.position_embedding(positions)

        return token_vectors + position_vectors
```

Перевіримо шар для пакета текстових послідовностей.

```
embedding_layer = TokenAndPositionEmbedding(
    max_len=MAX_LEN,
    vocab_size=len(vocab),
    d_model=64
)

for X_batch, _ in train_ds.take(1):
    embedded = embedding_layer(X_batch)

    print("Вхід:", X_batch.shape)
    print("Після Embedding:", embedded.shape)
```

Очікуваний результат:

```
Вхід: (32, 64)
Після Embedding: (32, 64, 64)
```

Отже, кожен із 64 токенів перетворюється на вектор із 64 числових компонентів.

7. Побудова моделі Transformer для класифікації

Основою моделі буде Transformer Encoder, реалізований у попередній частині прикладу. Використаємо два послідовні шари енкодера, після яких застосуємо Masked Mean Pooling і класифікаційний шар.

Реалізація моделі

```
class NewsTransformer(keras.Model):

    def __init__(
        self,
        vocab_size,
        max_len=64,
        d_model=64,
        num_heads=4,
```

```
        num_layers=2,
        ff_dim=128,
        num_classes=4,
        dropout=0.1
    ):
        super().__init__()

        # Ембеддинги токенів та позицій
        self.embedding = TokenAndPositionEmbedding(
            max_len=max_len,
            vocab_size=vocab_size,
            d_model=d_model
        )

        # Послідовність Encoder Blocks
        self.encoder_blocks = [
            TransformerEncoderBlock(
                d_model=d_model,
                num_heads=num_heads,
                ff_dim=ff_dim,
                dropout=dropout
            )
            for _ in range(num_layers)
        ]

        # Класифікаційна частина
        self.dropout = layers.Dropout(dropout)
        self.classifier = layers.Dense(num_classes)

    def call(
        self,
        token_ids,
        training=False,
        return_attention_scores=False
    ):

        # Маска справжніх токенів
        valid_tokens = tf.not_equal(token_ids, 0)

        # Маска для MultiHeadAttention
        attention_mask = tf.expand_dims(
            valid_tokens,
            axis=1
        )

        # Формування ембеддингів
        x = self.embedding(token_ids)

        all_attention_scores = []

        # Transformer Encoder
        for block in self.encoder_blocks:

            if return_attention_scores:

                x, scores = block(
                    x,
                    training=training,
                    attention_mask=attention_mask,
                    return_attention_scores=True
                )

                all_attention_scores.append(scores)

            else:

                x = block(
                    x,
                    training=training,
```

```
                attention_mask=attention_mask
            )

        # Masked Mean Pooling
        mask = tf.cast(
            valid_tokens[:, :, tf.newaxis],
            x.dtype
        )

        x = x * mask

        summed = tf.reduce_sum(x, axis=1)

        counts = tf.reduce_sum(mask, axis=1)
        counts = tf.maximum(counts, 1.0)

        pooled = summed / counts

        # Класифікація
        pooled = self.dropout(
            pooled,
            training=training
        )

        logits = self.classifier(pooled)

        if return_attention_scores:
            return logits, all_attention_scores

        return logits
```

Особливості реалізації. Параметр num_layers визначає кількість послідовних блоків енкодера, а num_heads – кількість голів уваги всередині кожного блоку.

Маска valid_tokens використовується двічі. Спочатку вона не дозволяє увазі враховувати доповнювальні токени як ключі та значення. Після проходження енкодера маска виключає доповнювальні позиції з операції Mean Pooling.

Створення моделі

```
model = NewsTransformer(
    vocab_size=len(vocab),
    max_len=MAX_LEN,
    d_model=64,
    num_heads=4,
    num_layers=2,
    ff_dim=128,
    num_classes=len(class_names),
    dropout=0.1
)

# Виклик для створення ваг
dummy_input = tf.ones(
    (1, MAX_LEN),
    dtype=tf.int32
)

_ = model(dummy_input)

model.summary()
```

Вихід моделі матиме розмірність $(B, 4)$, що відповідає чотирьом тематичним категоріям.

8. Налаштування навчання

Для оптимізації параметрів використаємо AdamW. Як функцію втрат застосуємо Sparse Categorical Cross-Entropy.

Оскільки вихідний шар формує логіти, встановлюємо from_logits=True.

```
model.compile(
    optimizer=keras.optimizers.AdamW(
        learning_rate=3e-4,
        weight_decay=1e-4
    ),

    loss=keras.losses.SparseCategoricalCrossentropy(
        from_logits=True
    ),

    metrics=[
        keras.metrics.SparseCategoricalAccuracy(
            name="accuracy"
        )
    ]
)
```

Для контролю перенавчання використаємо Early Stopping:

```
callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=2,
        restore_best_weights=True
    )
]
```

Навчання:

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10,
    callbacks=callbacks,
    verbose=1
)
```

Під час кожної епохи Keras відображатиме значення функції втрат та Accuracy для навчальної і валідаційної вибірок.

9. Аналіз процесу навчання

Після завершення навчання необхідно побудувати графіки зміни функції втрат і точності класифікації.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Функція втрат
axes[0].plot(
    history.history["loss"],
    label="Training Loss"
)

axes[0].plot(
    history.history["val_loss"],
    label="Validation Loss"
)

axes[0].set_title("Зміна функції втрат")
axes[0].set_xlabel("Епоха")
axes[0].set_ylabel("Cross-Entropy")
axes[0].legend()
axes[0].grid(True)

# Точність
axes[1].plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

axes[1].plot(
```

```
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

axes[1].set_title("Точність класифікації")
axes[1].set_xlabel("Епоха")
axes[1].set_ylabel("Accuracy")
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```

За зміни Cross-Entropy та Accuracy графіками необхідно оцінити, чи зменшується функція втрат, як змінюється точність класифікації та чи з'являються ознаки перенавчання.

10. Тестування моделі

Після навчання необхідно оцінити якість класифікації на тестовій вибірці, яка не використовувалася для оновлення параметрів моделі або вибору її гіперпараметрів.

```
test_loss, test_accuracy = model.evaluate(
    test_ds,
    verbose=1
)

print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
```

Для обчислення F1-score, Precision, Recall та побудови матриці помилок необхідно отримати прогнозовані класи всіх тестових текстів.

```
y_true = []
y_pred = []

for X_batch, y_batch in test_ds:

    logits = model(
        X_batch,
        training=False
    )

    predictions = tf.argmax(
        logits,
        axis=1
    )

    y_true.extend(y_batch.numpy())
    y_pred.extend(predictions.numpy())

y_true = np.array(y_true)
y_pred = np.array(y_pred)
```

Розрахунок метрик:

```
accuracy = accuracy_score(y_true, y_pred)

f1 = f1_score(
    y_true,
    y_pred,
    average="macro"
)

print(f"Accuracy: {accuracy:.4f}")
print(f"Macro-F1: {f1:.4f}")

print(
    classification_report(
        y_true,
        y_pred,
        labels=np.arange(len(class_names)),
```

```
        target_names=class_names,
        digits=4,
        zero_division=0
    )
)
```

У результаті буде сформовано звіт із Precision, Recall та F1-score для кожної з чотирьох тематичних категорій.

Числові результати залежать від випадкової ініціалізації, складу навчальної підмножини, кількості епох та інших параметрів. Тому їх необхідно отримати безпосередньо під час виконання лабораторної роботи.

10.1. Матриця помилок

Матриця помилок (Confusion Matrix) дозволяє визначити, які класи модель розпізнає найкраще та між якими категоріями виникає найбільше помилок.

```
cm = confusion_matrix(
    y_true,
    y_pred,
    labels=np.arange(len(class_names))
)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=class_names
)

fig, ax = plt.subplots(figsize=(7, 6))

disp.plot(
    ax=ax,
    cmap="Blues",
    values_format="d",
    colorbar=False
)

plt.xticks(rotation=30)
plt.title("Матриця помилок Transformer")
plt.tight_layout()
plt.show()
```

11. Візуалізація ваг багатоголової уваги

Для дослідження внутрішніх механізмів Transformer отримаємо матриці уваги для одного тестового документа.

Обрана реалізація моделі дозволяє повертати матриці ваг для кожного шару енкодера.

```
sample_text, sample_label = next(
    iter(test_raw.take(1))
)

sample_ids = vectorizer(
    tf.expand_dims(sample_text, axis=0)
)

logits, attention_maps = model(
    sample_ids,
    training=False,
    return_attention_scores=True
)

probabilities = tf.nn.softmax(
    logits,
    axis=-1
).numpy()[0]
```

```
predicted_class = np.argmax(probabilities)

print("Текст:")
print(sample_text.numpy().decode("utf-8"))

print("\nСправжній клас:",
      class_names[int(sample_label.numpy())])

print("Прогнозований клас:",
      class_names[predicted_class])

print("Імовірності:", probabilities)
```

Отримані матриці мають структуру:

$$A \in \mathbb{R}^{B \times H \times T \times T}.$$

У цьому прикладі для одного документа:

$$A \in \mathbb{R}^{1 \times 4 \times 64 \times 64}.$$

Кожен із двох блоків енкодера повертає чотири окремі матриці уваги.

11.1. Візуалізація голів останнього шару

Побудуємо теплові карти для чотирьох голів останнього шару Transformer.

Для зручності відображатимемо лише перші 20 дійсних токенів тексту.

```
# Індекси токенів одного документа
ids = sample_ids.numpy()[0]

# Кількість дійсних токенів для показу
n_tokens = min(
    20,
    int(np.sum(ids != 0))
)

tokens = [
    vocab[int(i)]
    for i in ids[:n_tokens]
]

# Матриці останнього шару
scores = attention_maps[-1].numpy()[0]

# Однакова шкала кольорів для всіх голів
vmax = max(
    float(np.max(scores[:, :n_tokens, :n_tokens])),
    1e-6
)

fig, axes = plt.subplots(
    2, 2,
    figsize=(14, 12)
)

for head, ax in enumerate(axes.flat):

    matrix = scores[
        head,
        :n_tokens,
        :n_tokens
    ]

    im = ax.imshow(
        matrix,
        cmap="Blues",
        vmin=0,
        vmax=vmax
    )
```

```
    ax.set_title(f"Голова уваги {head + 1}")

    ax.set_xticks(range(n_tokens))
    ax.set_xticklabels(
        tokens,
        rotation=90,
        fontsize=8
    )

    ax.set_yticks(range(n_tokens))
    ax.set_yticklabels(
        tokens,
        fontsize=8
    )

    ax.set_xlabel("Key")
    ax.set_ylabel("Query")

fig.colorbar(
    im,
    ax=axes.ravel().tolist(),
    shrink=0.65,
    label="Вага уваги"
)

plt.show()
```

Для візуалізації може бути показана лише частина матриці. Тому суми ваг у відображених рядках можуть бути меншими за одиницю, оскільки частина ваг припадає на інші токени, які не потрапили на графік.

11.2. Порівняння голів уваги

Для аналізу потрібно визначити, чи відрізняються структури матриць окремих голів.

Наприклад, можна перевірити, чи зосереджена увага певної голови на сусідніх токенах, чи спостерігається посилена взаємодія з окремими словами, чи розподіляються ваги приблизно рівномірно.

Також доцільно порівняти матриці першого й останнього шарів енкодера:

```
for layer_idx, layer_scores in enumerate(attention_maps):

    matrix = layer_scores.numpy()[
        0, 0, :n_tokens, :n_tokens
    ]

    plt.figure(figsize=(7, 6))

    plt.imshow(
        matrix,
        cmap="Blues"
    )

    plt.xticks(
        range(n_tokens),
        tokens,
        rotation=90
    )

    plt.yticks(
        range(n_tokens),
        tokens
    )

    plt.title(
        f"Encoder Layer {layer_idx + 1}, Head 1"
    )
```

```
    plt.colorbar(label="Вага уваги")
    plt.tight_layout()
    plt.show()
```

Результати дозволять дослідити, як змінюється структура ваг уваги між шарами Transformer.Водночас необхідно враховувати, що висока вага уваги не обов'язково означає визначальний вплив відповідного слова на кінцеве рішення класифікатора.

12. Порівняння Transformer із простішою моделлю

Для оцінювання доцільності використання Transformer рекомендується порівняти його з базовою моделлю, яка використовує Embedding та Global Average Pooling без механізму самоуваги.

```
baseline = keras.Sequential([
    layers.Input(shape=(MAX_LEN,)),

    layers.Embedding(
        input_dim=len(vocab),
        output_dim=64,
        mask_zero=True
    ),

    layers.GlobalAveragePooling1D(),

    layers.Dense(32, activation="relu"),

    layers.Dropout(0.1),

    layers.Dense(len(class_names))
])

baseline.compile(
    optimizer=keras.optimizers.AdamW(
        learning_rate=3e-4,
        weight_decay=1e-4
    ),

    loss=keras.losses.SparseCategoricalCrossentropy(
        from_logits=True
    ),

    metrics=["accuracy"]
)

baseline.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10,
    callbacks=[
        keras.callbacks.EarlyStopping(
            monitor="val_loss",
            patience=2,
            restore_best_weights=True
        )
    ],
    verbose=1
)

baseline_loss, baseline_acc = baseline.evaluate(
    test_ds,
    verbose=0
)

print("Baseline Accuracy:", baseline_acc)
print("Transformer Accuracy:", test_accuracy)
```

Обидві моделі навчаються на однакових навчальних і валідаційних даних та оцінюються на тій самій тестовій вибірці.

Результати можна подати в такій таблиці:

Модель	Accuracy	Macro-F1	Особливість
Embedding + GlobalAveragePooling	Обчислити	Обчислити	Без механізму уваги
Transformer Encoder	Обчислити	Обчислити	Multi-Head Self-Attention

Для заповнення Macro-F1 базової моделі потрібно отримати її прогнози за аналогією з розділом 10.

Порівняння дозволяє з'ясувати, чи забезпечує використання самоуваги покращення класифікації саме для досліджуваних даних і налаштувань.

13. Аналіз результатів прикладу

Під час виконання необхідно проаналізувати результати навчання, точність і F1-score на тестовій вибірці, матрицю помилок, а також структуру ваг уваги для вибраних документів.

Особливу увагу слід приділити порівнянню різних голів уваги та оцінюванню того, наскільки їхня структура узгоджується зі змістом текстових послідовностей.

Примітка. Наведений код є цілісним навчальним прикладом для використання разом із TransformerEncoderBlock із попередньої частини. Фактичні графіки й числові метрики формуються після його запуску.

Приклад. Прогнозування часових рядів за допомогою Transformer

Частина 3. Побудова та навчання Transformer для прогнозування технологічних показників

Завдання. Побудувати модель Transformer Encoder засобами TensorFlow/Keras для прогнозування значень технологічного показника на основі багатовимірних часових послідовностей. Сформувати синтетичний набір даних, що імітує роботу технологічного обладнання, виконати стандартизацію даних і формування часових вікон. Реалізувати модель Transformer для регресії, виконати її навчання та оцінити точність прогнозування за метриками MAE, RMSE і R^2. Візуалізувати фактичні та прогнозовані значення, дослідити матриці багатоголової уваги й проаналізувати вплив історичних спостережень на результати моделі.

У прикладі використовуються чотири взаємопов'язані показники технологічного процесу:

- **Температура** – temperature, °C.
- **Тиск** – pressure, бар.
- **Швидкість обертання** – speed, об/хв.
- **Вібрація** – vibration, умовні одиниці.

Метою прогнозування є визначення температури на наступному часовому кроці на основі попередніх 48 вимірювань усіх чотирьох показників.

Формально задача визначається як

$$\hat{T}_{t+1} = f_\theta(X_{t-47}, X_{t-46}, \dots, X_t),$$

де

$$X_t = [T_t, P_t, S_t, V_t],$$

T_t – температура, P_t – тиск, S_t – швидкість обертання, V_t – вібрація, f_θ – модель Transformer.

У цьому випадку маємо задачу **багатовимірного однокрокового прогнозування (Multivariate One-Step Forecasting)**.

1. Підключення бібліотек

Для реалізації використаємо TensorFlow/Keras, NumPy, Pandas, Matplotlib та scikit-learn.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow import keras
from tensorflow.keras import layers

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

np.random.seed(42)
tf.random.set_seed(42)

print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices("GPU"))
```

У прикладі передбачається використання класу TransformerEncoderBlock, реалізованого в попередній частині лабораторної роботи.

2. Формування синтетичного набору технологічних даних

Сформуємо багатовимірний часовий ряд із 4000 спостережень.Для імітації технологічного процесу показники повинні містити часові залежності, періодичні коливання та випадкові відхилення. Крім того, між окремими показниками встановимо взаємозв'язки.Наприклад, зміна швидкості обертання впливатиме на тиск і вібрацію, а температурний режим залежатиме від попередньої температури, швидкості обертання та тиску.Такі дані дозволять дослідити, чи здатна модель Transformer виявляти залежності між різними часовими позиціями та параметрами процесу.

Генерація даних

```
N = 4000

rng = np.random.default_rng(42)

time = np.arange(N)

temperature = np.zeros(N)
pressure = np.zeros(N)
speed = np.zeros(N)
vibration = np.zeros(N)

# Початкові значення
temperature[0] = 65.0
pressure[0] = 5.0
speed[0] = 1500.0
vibration[0] = 0.5

for t in range(1, N):

    # Періодична зміна швидкості
    speed[t] = (
```

```
        1500
        + 180 * np.sin(2 * np.pi * t / 120)
        + 65 * np.sin(2 * np.pi * t / 35)
        + rng.normal(0, 12)
    )

    # Тиск залежить від попереднього
    # значення та швидкості
    pressure[t] = (
        0.75 * pressure[t-1]
        + 0.25 * (
            5.0 + 0.002 * (speed[t] - 1500)
        )
        + rng.normal(0, 0.03)
    )

    # Температурний режим
    temperature[t] = (
        0.88 * temperature[t-1]
        + 0.12 * (
            65
            + 0.012 * (speed[t-1] - 1500)
            + 1.4 * (pressure[t-1] - 5)
            + 2 * np.sin(2 * np.pi * t / 200)
        )
        + rng.normal(0, 0.18)
    )

    # Вібрація залежить від швидкості
    # та попереднього рівня вібрації
    vibration[t] = (
        0.65 * vibration[t-1]
        + 0.35 * (
            0.5
            + 0.0015 * abs(speed[t] - 1500)
        )
        + rng.normal(0, 0.015)
    )
```

У цьому прикладі температура має інерційний характер: її поточне значення залежить від попереднього стану. Взаємозв'язки між показниками задані штучно, тому отримані залежності не слід вважати фізичною моделлю конкретного обладнання.

Формування DataFrame

```
data = pd.DataFrame({
    "temperature": temperature,
    "pressure": pressure,
    "speed": speed,
    "vibration": vibration
})

print(data.head())
print(data.describe())
```

У результаті формується таблиця з чотирма числовими показниками для 4000 послідовних моментів часу.

3. Візуалізація початкових часових рядів

Перед побудовою моделі необхідно проаналізувати структуру даних. Для цього побудуємо графіки всіх чотирьох технологічних показників.

```
fig, axes = plt.subplots(
    4, 1,
    figsize=(12, 10),
    sharex=True
)
```

```
columns = [
    "temperature",
    "pressure",
    "speed",
    "vibration"
]

titles = [
    "Температура, °C",
    "Тиск, бар",
    "Швидкість обертання, об/хв",
    "Вібрація, ум. од."
]

for i, column in enumerate(columns):

    axes[i].plot(
        data[column].values,
        linewidth=1
    )

    axes[i].set_title(titles[i])
    axes[i].set_ylabel("Значення")
    axes[i].grid(True)

axes[-1].set_xlabel("Часовий крок")

plt.tight_layout()
plt.show()
```

За графіками необхідно проаналізувати наявність періодичних коливань, випадкових відхилень, інерційності та можливих зв’язків між показниками.

4. Розподіл даних і стандартизація

Для коректного оцінювання прогнозування виконаємо хронологічний поділ даних.

Використаємо таке співвідношення:

Вибірка	Кількість початкових спостережень	Частка
Навчальна	2800	70%
Валідаційна	600	15%
Тестова	600	15%

На відміну від задачі класифікації текстів, часові ряди не можна випадково перемішувати перед поділом, оскільки необхідно зберегти їхню хронологічну структуру.

```
train_end = 2800
val_end = 3400

values = data.values.astype(np.float32)

scaler = StandardScaler()

# Параметри визначаються лише
# за навчальною частиною
scaler.fit(values[:train_end])

scaled_values = scaler.transform(values).astype(
    np.float32
)

print("Форма даних:", scaled_values.shape)
```

Стандартизація виконується за формулою

$$x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}},$$

де μ_{train} та σ_{train} – середнє значення і стандартне відхилення відповідного показника, обчислені лише за навчальною частиною.

Важливо: для валідаційних і тестових даних використовуються ті самі параметри стандартизації, що були визначені на навчальній вибірці. Це запобігає витоку інформації з майбутніх спостережень.

5. Формування часових вікон

Transformer отримуватиме на вході історичну послідовність із 48 часових кроків. За нею модель прогнозуватиме температуру на наступному кроці.

Для кожної позиції t:

$$X_t = \begin{bmatrix} x_{t-47} \\ x_{t-46} \\ \vdots \\ x_t \end{bmatrix} \in \mathbb{R}^{48\times 4},$$

а цільове значення:

$$y_t = T_{t+1}.$$

Формування послідовностей реалізуємо методом ковзного вікна.

```
SEQ_LEN = 48

TARGET_COL = 0  # temperature

def create_sequences(
    values,
    seq_len=48,
    target_col=0
):

    X = []
    y = []
    target_indices = []

    for target_idx in range(seq_len, len(values)):

        # Історичне вікно до моменту target_idx
        window = values[
            target_idx - seq_len : target_idx
        ]

        # Температура на наступному кроці
        target = values[target_idx, target_col]

        X.append(window)
        y.append(target)
        target_indices.append(target_idx)

    return (
        np.asarray(X, dtype=np.float32),
        np.asarray(y, dtype=np.float32),
        np.asarray(target_indices)
    )


X_all, y_all, target_indices = create_sequences(
    scaled_values,
    seq_len=SEQ_LEN,
    target_col=TARGET_COL
)
```

Для запобігання витоку інформації розподілятимемо сформовані вікна за індексом цільового значення.

```
train_mask = target_indices < train_end

val_mask = (
    (target_indices >= train_end)
    & (target_indices < val_end)
)

test_mask = target_indices >= val_end

X_train = X_all[train_mask]
y_train = y_all[train_mask]

X_val = X_all[val_mask]
y_val = y_all[val_mask]

X_test = X_all[test_mask]
y_test = y_all[test_mask]
```

У такому підході історичне вікно на початку валідаційної або тестової вибірки може містити попередні спостереження з іншої частини даних. Це припустимо для однокрокового прогнозування, оскільки на момент прогнозу ці значення вже відомі.

Головна умова полягає в тому, що вхідне вікно не повинно містити майбутнього цільового значення.

Перевірка розмірностей

```
print("X_train:", X_train.shape)
print("y_train:", y_train.shape)

print("X_val:", X_val.shape)
print("y_val:", y_val.shape)

print("X_test:", X_test.shape)
print("y_test:", y_test.shape)
```

Очікувані розмірності:

```
X_train: (2752, 48, 4)
y_train: (2752,)

X_val: (600, 48, 4)
y_val: (600,)

X_test: (600, 48, 4)
y_test: (600,)
```

Кожен вхідний об’єкт є матрицею розмірності 48×4, яка містить історію чотирьох технологічних параметрів за 48 часових кроків.

6. Побудова моделі Transformer для прогнозування

Побудуємо модель на основі Transformer Encoder. На відміну від класифікації текстів, на вхід подаватимуться не індекси токенів, а числові вектори технологічних параметрів.

Для перетворення чотирьох вхідних ознак у простір розмірності d_{model} використаємо лінійнумодель:

$$E_t = W_e x_t + b_e.$$

До отриманих векторів додається навчуване позиційне кодування. Далі сформована послідовність проходить через два шари Transformer Encoder.

Вихідне представлення останньої часової позиції використовується для однокрокового прогнозування:

$$\hat{T}_{t+1} = W_p H_t + b_p,$$

де H_t – контекстне представлення останнього елемента історичного вікна.

6.1. Реалізація моделі

Використаємо TransformerEncoderBlock, реалізований у першій частині практичного прикладу.

```
class TimeSeriesTransformer(keras.Model):

    def __init__(
        self,
        seq_len=48,
        num_features=4,
        d_model=64,
        num_heads=4,
        num_layers=2,
        ff_dim=128,
        dropout=0.1
    ):
        super().__init__()

        self.seq_len = seq_len

        # Перетворення числових ознак
        self.input_projection = layers.Dense(d_model)

        # Навчуване позиційне кодування
        self.position_embedding = layers.Embedding(
            input_dim=seq_len,
            output_dim=d_model
        )

        # Шари Transformer Encoder
        self.encoder_blocks = [
            TransformerEncoderBlock(
                d_model=d_model,
                num_heads=num_heads,
                ff_dim=ff_dim,
                dropout=dropout
            )
            for _ in range(num_layers)
        ]

        # Вихідна регресійна частина
        self.dropout = layers.Dropout(dropout)
        self.regressor = layers.Dense(1)

    def call(
        self,
        inputs,
        training=False,
        return_attention_scores=False
    ):

        # Лінійна проєкція
        x = self.input_projection(inputs)

        # Позиційні ембеддинги
        seq_len = tf.shape(inputs)[1]

        positions = tf.range(seq_len)

        pos_encoding = self.position_embedding(
            positions
        )

        x = x + pos_encoding

        all_attention_scores = []

        # Transformer Encoder
        for block in self.encoder_blocks:
```

```
            if return_attention_scores:

                x, scores = block(
                    x,
                    training=training,
                    return_attention_scores=True
                )

                all_attention_scores.append(scores)

            else:

                x = block(
                    x,
                    training=training
                )

        # Представлення останнього моменту часу
        x = x[:, -1, :]

        x = self.dropout(
            x,
            training=training
        )

        # Прогноз одного значення
        prediction = self.regressor(x)

        if return_attention_scores:
            return prediction, all_attention_scores

        return prediction
```

У моделі використовується шар Dense(d_model), який перетворює кожен вхідний вектор із чотирьох ознак на 64-вимірне представлення.Позиційні ембеддинги дозволяють моделі враховувати порядок спостережень у часовому вікні.Блоки Transformer Encoder виконують обчислення багатоголової уваги та нелінійні перетворення представлень. Після цього використовується остання часова позиція, яка внаслідок самоуваги може містити інформацію з усього історичного вікна.Вихідний шар Dense(1) формує одне числове значення – прогноз стандартизованої температури наступного часового кроку.

6.2. Створення моделі та перевірка розмірностей

```
model = TimeSeriesTransformer(
    seq_len=SEQ_LEN,
    num_features=4,
    d_model=64,
    num_heads=4,
    num_layers=2,
    ff_dim=128,
    dropout=0.1
)

# Перевірка на одному пакеті
X_sample = tf.convert_to_tensor(
    X_train[:8],
    dtype=tf.float32
)

predictions, attention_maps = model(
    X_sample,
    return_attention_scores=True
)

print("Вхід:", X_sample.shape)
print("Прогноз:", predictions.shape)
```

```
for i, attention in enumerate(attention_maps):

    print(
        f"Encoder {i+1}:",
        attention.shape
    )

model.summary()
```

Очікувані розмірності:

```
Вхід: (8, 48, 4)
Прогноз: (8, 1)

Encoder 1: (8, 4, 48, 48)
Encoder 2: (8, 4, 48, 48)
```

Кожен енкодер формує чотири матриці уваги розміром 48×48 для кожної послідовності пакета.Це дозволяє окремо досліджувати, як модель розподіляє увагу між різними часовими позиціями.

7. Підготовка даних до навчання TensorFlow

Для ефективного навчання перетворимо NumPy-масиви на об’єкти tf.data.Dataset.

```
BATCH_SIZE = 64
AUTOTUNE = tf.data.AUTOTUNE

def prepare_dataset(X, y, training=False):

    dataset = tf.data.Dataset.from_tensor_slices(
        (X, y)
    )

    if training:
        dataset = dataset.shuffle(
            buffer_size=len(X),
            seed=42
        )

    dataset = dataset.batch(BATCH_SIZE)
    dataset = dataset.prefetch(AUTOTUNE)

    return dataset


train_ds = prepare_dataset(
    X_train, y_train, training=True
)

val_ds = prepare_dataset(X_val, y_val)

test_ds = prepare_dataset(X_test, y_test)
```

Операція shuffle() застосовується лише до сформованих навчальних вікон. Вона змінює порядок подачі навчальних прикладів у пакети, але не порушує порядок вимірювань усередині кожного вікна.Валідаційна й тестова вибірки не перемішуються.

8. Навчання Transformer

Оскільки прогнозується числовий показник, використовуватимемо функцію втрат Mean Squared Error:

$$\mathcal{L}_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2.$$

Для оптимізації параметрів застосуємо AdamW.

```
model.compile(
    optimizer=keras.optimizers.AdamW(
        learning_rate=3e-4,
        weight_decay=1e-4
    ),

    loss="mse",

    metrics=[
        keras.metrics.MeanAbsoluteError(
            name="mae"
        )
    ]
)
```

Для запобігання перенавчанню використаємо EarlyStopping, а для адаптивного зменшення швидкості навчання – ReduceLROnPlateau.

```
callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=5,
        restore_best_weights=True
    ),

    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=3,
        min_lr=1e-6
    )
]
```

Навчання моделі:

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=30,
    callbacks=callbacks,
    verbose=1
)
```

Під час навчання для кожної епохи відображатимуться значення MSE та MAE на навчальній і валідаційній вибірках.Оскільки цільові значення стандартизовані, ці метрики на етапі навчання також обчислюються у стандартизованому масштабі.

9. Аналіз процесу навчання

Для оцінювання збіжності моделі побудуємо графіки зміни функції втрат , MSE та MAE.

```
fig, axes = plt.subplots(
    1, 2,
    figsize=(12, 4)
)

# MSE
axes[0].plot(
    history.history["loss"],
    label="Training MSE"
)

axes[0].plot(
    history.history["val_loss"],
    label="Validation MSE"
)

axes[0].set_title("Зміна функції втрат")
axes[0].set_xlabel("Епоха")
axes[0].set_ylabel("MSE")
axes[0].legend()
```

```
axes[0].grid(True)

# MAE
axes[1].plot(
    history.history["mae"],
    label="Training MAE"
)

axes[1].plot(
    history.history["val_mae"],
    label="Validation MAE"
)

axes[1].set_title("Середня абсолютна похибка")
axes[1].set_xlabel("Епоха")
axes[1].set_ylabel("MAE")
axes[1].legend()
axes[1].grid(True)

plt.tight_layout()
plt.show()
```

Під час аналізу необхідно визначити, чи зменшується помилка прогнозування, чи спостерігається перенавчання, на якій епосі досягнуто найменшої валідаційної похибки та чи було використано дострокове припинення навчання.

10. Прогнозування на тестовій вибірці

Після навчання використаємо модель для прогнозування температури на тестовій вибірці.

```
y_pred_scaled = model.predict(
    test_ds,
    verbose=1
).reshape(-1)

y_true_scaled = y_test
```

Оскільки під час навчання застосовувалася стандартизація, прогнозовані значення необхідно повернути до початкового масштабу температури:

$$T = \hat{T}_{std} \cdot \sigma_T + \mu_T.$$

Тут σ_T і μ_T – параметри стандартизації температури, отримані лише з навчальної вибірки.

```
temp_mean = scaler.mean_[TARGET_COL]
temp_std = scaler.scale_[TARGET_COL]

y_pred = (
    y_pred_scaled * temp_std
    + temp_mean
)

y_true = (
    y_true_scaled * temp_std
    + temp_mean
)
```

Тепер значення y_pred і y_true виражені в градусах Цельсія.

10.1. Обчислення метрик прогнозування

```
mae = mean_absolute_error(
    y_true, y_pred
)

rmse = np.sqrt(
    mean_squared_error(y_true, y_pred)
)
```

```
r2 = r2_score(
    y_true, y_pred
)

print(f"MAE: {mae:.4f} °C")
print(f"RMSE: {rmse:.4f} °C")
print(f"R²: {r2:.4f}")
```

11. Порівняння з наївним прогнозуванням

Для оцінювання ефективності Transformer необхідно порівняти його з простішим базовим підходом.У задачах однокрокового прогнозування часових рядів часто використовують наївний прогноз (Persistence Forecast), за якого майбутнє значення вважається рівним останньому відомому:

$$\hat{T}_{t+1}^{naive} = T_t.$$

Цей метод не потребує навчання і є корисною базою для порівняння.

Реалізація

```
# Остання відома температура
# в кожному тестовому вікні
naive_scaled = X_test[:, -1, TARGET_COL]

naive_pred = (
    naive_scaled * temp_std
    + temp_mean
)

naive_mae = mean_absolute_error(
    y_true, naive_pred
)

naive_rmse = np.sqrt(
    mean_squared_error(y_true, naive_pred)
)

naive_r2 = r2_score(
    y_true, naive_pred
)

print(f"Naive MAE: {naive_mae:.4f} °C")
print(f"Naive RMSE: {naive_rmse:.4f} °C")
print(f"Naive R²: {naive_r2:.4f}")
```

Порівняльна таблиця

```
comparison = pd.DataFrame({
    "Model": ["Naive", "Transformer"],
    "MAE": [naive_mae, mae],
    "RMSE": [naive_rmse, rmse],
    "R2": [naive_r2, r2]
})

print(comparison.round(4))
```

Додатково обчислимо відносну зміну RMSE порівняно з наївним методом:

$$Improvement_{RMSE} = \frac{RMSE_{naive} - RMSE_{Transformer}}{RMSE_{naive}} \cdot 100\%.$$

```
improvement = (
    (naive_rmse - rmse)
    / naive_rmse
    * 100
)

print(f"Зміна RMSE: {improvement:.2f}%")
```

Додатне значення свідчить про покращення RMSE порівняно з Naive, від'ємне – про погіршення.Якщо синтетична температура має виражену інерційність, то наївний прогноз може бути досить точним. Transformer не обов'язково його перевершить, і такий результат також потрібно проаналізувати.

12. Візуалізація результатів прогнозування

12.1. Порівняння фактичних і прогнозованих значень

Побудуємо графік для перших 200 спостережень тестової вибірки.

```
n_plot = 200

plt.figure(figsize=(13, 5))

plt.plot(
    y_true[:n_plot],
    label="Фактична температура",
    linewidth=2
)

plt.plot(
    y_pred[:n_plot],
    label="Transformer",
    linewidth=1.5
)

plt.plot(
    naive_pred[:n_plot],
    label="Naive",
    linestyle="--",
    alpha=0.7
)

plt.xlabel("Тестовий часовий крок")
plt.ylabel("Температура, °C")
plt.title("Прогнозування температури")

plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

Графік дозволяє оцінити, наскільки модель відтворює характер зміни температури, чи правильно прогнозує локальні підвищення та зниження, а також чи спостерігається систематичне відставання прогнозу від фактичних значень.

12.2. Аналіз похибок прогнозування

Похибка для кожного тестового спостереження визначається як

$$e_i = y_i - \hat{y}_i.$$

```
errors = y_true - y_pred

fig, axes = plt.subplots(
    1, 2,
    figsize=(12, 4)
)

axes[0].plot(errors)
axes[0].axhline(0, linestyle="--")
axes[0].set_title("Похибки прогнозування")
axes[0].set_xlabel("Тестовий часовий крок")
axes[0].set_ylabel("Похибка, °C")
axes[0].grid(True)
```

```
axes[1].hist(
    errors,
    bins=30,
    edgecolor="black"
)

axes[1].set_title("Розподіл похибок")
axes[1].set_xlabel("Похибка, °C")
axes[1].set_ylabel("Частота")

plt.tight_layout()
plt.show()
```

За результатами необхідно оцінити, чи мають похибки систематичне зміщення, чи змінюється їхня величина протягом тестового періоду та чи виникають окремі значні відхилення прогнозу.

13. Візуалізація ваг уваги Transformer для часових рядів

Механізм багатоголової уваги дозволяє проаналізувати, як модель формує залежності між різними часовими позиціями історичного вікна.У розглянутій моделі використовуються два шари Transformer Encoder, кожен із яких містить чотири голови уваги.

Для вхідної послідовності довжини 48 кожна голова формує матрицю:

$$A^{(h)} \in \mathbb{R}^{48\times 48}.$$

Елемент $a_{ij}^{(h)}$ характеризує вагу, з якою позиція j враховується під час формування представлення позиції i у відповідній голові.

13.1. Отримання матриць уваги

Виберемо один приклад із тестової вибірки.

```
sample_idx = 100

sample = tf.convert_to_tensor(
    X_test[sample_idx:sample_idx+1],
    dtype=tf.float32
)

prediction, attention_maps = model(
    sample,
    training=False,
    return_attention_scores=True
)

for i, attention in enumerate(attention_maps):
    print(
        f"Encoder {i+1}:",
        attention.shape
    )
```

Очікувані розмірності:

```
Encoder 1: (1, 4, 48, 48)
Encoder 2: (1, 4, 48, 48)
```

13.2. Побудова теплових карт

Для кожної голови останнього шару побудуємо окрему теплову карту.

```
scores = attention_maps[-1].numpy()[0]

fig, axes = plt.subplots(
    2, 2,
    figsize=(12, 10)
)
```

```
for head, ax in enumerate(axes.flat):

    matrix = scores[head]

    im = ax.imshow(
        matrix,
        cmap="Blues",
        vmin=0,
        vmax=1
    )

    ax.set_title(f"Голова {head + 1}")

    ax.set_xlabel("Часова позиція Key")
    ax.set_ylabel("Часова позиція Query")

fig.colorbar(
    im,
    ax=axes.ravel().tolist(),
    shrink=0.65,
    label="Вага уваги"
)

plt.show()
```

Під час аналізу необхідно звернути увагу на характер розподілу ваг уздовж головної діагоналі, наявність підвищених ваг для окремих часових позицій та відмінності між головами.

13.3. Аналіз уваги останньої часової позиції

Оскільки для прогнозування використовується вихідне представлення останнього елемента послідовності, доцільно окремо проаналізувати, як цей елемент розподіляє увагу між усіма 48 історичними позиціями.

Для останнього шару:

$$a_{L,j}^{(h)}, \qquad j = 1, \dots, L,$$

де $L = 48$, h – номер голови.

```
last_query_attention = scores[:, -1, :]

plt.figure(figsize=(12, 5))

for head in range(4):

    plt.plot(
        np.arange(SEQ_LEN),
        last_query_attention[head],
        label=f"Голова {head + 1}"
    )

plt.xlabel("Позиція в історичному вікні")
plt.ylabel("Вага уваги")

plt.title(
    "Увага останньої позиції до історичних спостережень"
)

plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Графік ваг уваги останньої часової позиції за чотирма головами Transformer дозволяє дослідити, чи приділяє остання позиція більше уваги найближчим або віддаленим спостереженням. Водночас він не показує повного впливу попередніх позицій на кінцевий

прогноз, оскільки інформація проходить через кілька шарів, залишкові з'єднання та лінійні перетворення.

14. Додаткова перевірка інтерпретованості методом Occlusion

Для доповнення аналізу уваги застосуємо метод Occlusion – послідовну зміну окремих часових позицій вхідного вікна.У цьому прикладі кожну часову позицію тимчасово замінюватимемо вектором нульових стандартизованих значень. Нуль у стандартизованому просторі відповідає середньому значенню відповідної ознаки навчальної вибірки.

Після такої заміни повторно обчислимо прогноз температури.

Величину зміни прогнозу визначимо як

$$\Delta_j = \left|\hat{T}_{t+1} - \hat{T}_{t+1}^{(j)}\right|,$$

де $\hat{T}_{t+1}^{(j)}$ – прогноз після зміни вхідних значень на позиції j.

```
original = X_test[sample_idx:sample_idx+1].copy()

base_prediction = model(
    original,
    training=False
).numpy().reshape(-1)[0]

occlusion_effect = []

for pos in range(SEQ_LEN):

    modified = original.copy()

    # Замінюємо всі чотири ознаки
    # на певній часовій позиції
    modified[0, pos, :] = 0.0

    new_prediction = model(
        modified,
        training=False
    ).numpy().reshape(-1)[0]

    # Переведення зміни прогнозу в °C
    difference = (
        abs(base_prediction - new_prediction)
        * temp_std
    )

    occlusion_effect.append(difference)
```

Візуалізація:

```
plt.figure(figsize=(12, 4))

plt.bar(
    np.arange(SEQ_LEN),
    occlusion_effect
)

plt.xlabel("Позиція в історичному вікні")
plt.ylabel("Зміна прогнозу, °C")

plt.title(
    "Чутливість прогнозу до зміни часових позицій"
)

plt.grid(axis="y")
plt.tight_layout()
plt.show()
```

15. Аналіз отриманих результатів

У розглянутому прикладі модель Transformer використовується для прогнозування температури технологічного процесу за історією чотирьох вимірюваних параметрів.

Після виконання програмного коду необхідно зіставити результати моделі Transformer із наївним прогнозуванням, проаналізувати динаміку навчання, графіки фактичних і прогнозованих значень та особливості розподілу ваг уваги.

Результати експерименту рекомендується узагальнити в таблиці.

Характеристика	Naive	Transformer
MAE, °C	Обчислити	Обчислити
RMSE, °C	Обчислити	Обчислити
R^2	Обчислити	Обчислити
Потребує навчання	Ні	Так
Використовує всі чотири показники	Ні	Так
Використовує механізм самоуваги	Ні	Так

Потрібно встановити, чи забезпечує Transformer покращення точності прогнозування порівняно з Naive, та оцінити, наскільки виправдане використання складнішої нейромережевої архітектури для досліджуваного часового ряду.

Окремо слід проаналізувати матриці ваг уваги різних голів, визначити особливості їхнього розподілу та порівняти результати візуалізації із чутливістю прогнозу до зміни окремих часових позицій.

За результатами необхідно сформулювати висновки щодо здатності Transformer моделювати часові залежності, точності однокрокового прогнозування та можливостей інтерпретації механізму багатоголової уваги.

Хід роботи

1. **Реалізувати механізм Self-Attention** засобами TensorFlow. Для тестової послідовності обчислити матриці Query, Key та Value, оцінки подібності, нормалізовані ваги уваги й вихідні представлення. Перевірити нормалізацію ваг і побудувати теплову карту матриці уваги.

2. **Реалізувати механізм Multi-Head Attention.** Створити власний шар багатоголової уваги та порівняти його з tf.keras.layers.MultiHeadAttention. Перевірити розмірності тензорів, отримати матриці окремих голів і побудувати їхні теплові карти.

3. **Побудувати блок Transformer Encoder**, що містить Multi-Head Attention, Feed-Forward Network, залишкові з'єднання, Layer Normalization і Dropout. Передбачити можливість отримання ваг уваги для аналізу.

4. **Вибрати прикладне завдання відповідно до індивідуального варіанта:**

   – Варіанти 1–10 – класифікація текстів на наборі даних AG News. Виконати токенізацію, сформувати словник і послідовності індексів токенів, реалізувати Embedding та позиційне кодування.

   – Варіанти 11–20 – прогнозування часових рядів технологічних показників. Сформувати часові ряди температури, тиску, швидкості обертання та вібрації, виконати стандартизацію й підготувати історичні вікна методом ковзного вікна.

5. **Побудувати прикладну модель Transformer.** Для класифікації текстів використати Transformer Encoder із класифікаційним вихідним шаром. Для прогнозування часового ряду – Transformer Encoder із регресійним вихідним шаром.

6. **Навчити модель** із використанням оптимізатора AdamW. Для класифікації використати Sparse Categorical Cross-Entropy, для прогнозування – MSE. Побудувати графіки зміни функції втрат і відповідних метрик на навчальній та валідаційній вибірках.

7. **Оцінити результати на тестовій вибірці.** Для класифікації обчислити Accuracy, Precision, Recall і Macro-F1, побудувати матрицю помилок. Для прогнозування обчислити MAE, RMSE і R^2, побудувати графіки фактичних і прогнозованих значень та похибок прогнозування.

8. **Порівняти Transformer із базовою моделлю.** Для класифікації використати модель Embedding + GlobalAveragePooling, для прогнозування – наївний прогноз, за якого наступне значення дорівнює останньому відомому. Порівняти результати на однаковій тестовій вибірці.

9. **Дослідити ваги уваги навченої моделі.** Отримати матриці ваг окремих голів і шарів Transformer, побудувати теплові карти та проаналізувати особливості розподілу уваги між токенами або часовими позиціями.

10. **Виконати індивідуальне дослідження** відповідно до номера варіанта. Для досліджуваного параметра використати не менше трьох значень або конфігурацій, якщо інше не передбачено варіантом. Порівняти результати, побудувати графіки та сформулювати висновки щодо впливу параметра на якість моделі.

Індивідуальні завдання

Усі студенти виконують базову реалізацію механізмів Self-Attention та Multi-Head Attention. Відповідно до номера варіанта будують модель Transformer для одного типу послідовних даних і виконують додаткове дослідження.

Варіанти 1–10. Класифікація текстів

Для виконання використовується набір даних AG News, що містить чотири тематичні категорії текстів. Як базову модель застосувати Transformer Encoder із двома шарами, чотирма головами уваги та розмірністю представлення 64.

№	Завдання	Що змінюється	Що необхідно порівняти
1	Дослідити вплив кількості голів уваги	1, 2, 4, 8 голів	Accuracy, Macro-F1, час навчання, матриці уваги
2	Дослідити вплив кількості шарів Transformer Encoder	1, 2, 3 шари	Accuracy, Macro-F1, збіжність навчання, кількість параметрів
3	Дослідити вплив розмірності ембеддингів	32, 64, 128	Accuracy, Macro-F1, кількість параметрів, час навчання
4	Дослідити вплив розмірності Feed-Forward Network	64, 128, 256 нейронів	Accuracy, Macro-F1, швидкість і стабільність навчання
5	Дослідити вплив довжини текстової послідовності	32, 64, 128 токенів	Accuracy, Macro-F1, частку обрізаних текстів, час навчання
6	Порівняти способи позиційного кодування	Без кодування, синусоїдальне, навчуване	Accuracy, Macro-F1, матриці уваги
7	Порівняти способи агрегування представлень	Masked Mean Pooling, Masked Max Pooling, Last Valid Token	Accuracy, Macro-F1, помилки класифікації
8	Дослідити вплив кількості навчальних даних	1000, 4000, 8000 текстів	Accuracy, Macro-F1, залежність якості від обсягу вибірки
9	Дослідити вплив регуляризації Dropout	0.0, 0.1, 0.3	Навчальну й валідаційну похибки, Accuracy, Macro-F1
10	Дослідити інтерпретованість механізму уваги	Ваги уваги та Occlusion для 12 текстів різних класів	Теплові карти, зміну ймовірностей класів після заміни токенів

Варіанти 11–20. Прогнозування часових рядів

Для виконання використовується синтетичний багатовимірний часовий ряд технологічних показників: температура, тиск, швидкість обертання та вібрація. Цільовим показником є температура. Базова модель використовує історичне вікно з 48 спостережень і виконує однокрокове прогнозування.

№	Завдання	Що змінюється	Що необхідно порівняти
11	Дослідити вплив кількості голів уваги	1, 2, 4, 8 голів	MAE, RMSE, R^2, матриці уваги
12	Дослідити вплив кількості шарів Transformer Encoder	1, 2, 3 шари	MAE, RMSE, R^2, час навчання
13	Дослідити вплив довжини історичного вікна	24, 48, 96 спостережень	MAE, RMSE, R^2, увагу до віддалених позицій
14	Дослідити вплив горизонту прогнозування	1, 3, 7 часових кроків	MAE, RMSE, R^2, зміну похибки з горизонтом
15	Дослідити вплив кількості вхідних ознак	Температура; температура й тиск; усі 4 показники	MAE, RMSE, R^2, вплив додаткових ознак
16	Порівняти способи позиційного кодування	Без кодування, синусоїдальне, навчуване	MAE, RMSE, R^2, матриці уваги
17	Дослідити вплив розмірності ембеддингів	32, 64, 128	MAE, RMSE, R^2, кількість параметрів
18	Дослідити стійкість моделі до шуму	Стандартне відхилення додаткового шуму: 0, 0.1, 0.2	MAE, RMSE, R^2, стійкість прогнозу
19	Дослідити вплив регуляризації Dropout	0.0, 0.1, 0.3	Навчальну й валідаційну MSE, тестові MAE та RMSE
20	Дослідити інтерпретованість ваг уваги для часових рядів	Три тестові вікна з різними температурними режимами	Матриці уваги, Occlusion, чутливість прогнозу до часових позицій

Вимоги до експериментального порівняння. Для кожної конфігурації необхідно використовувати однакові вибірки та фіксувати параметри навчання, які не досліджуються. Для задач із різною довжиною історичного вікна або горизонтом прогнозування потрібно забезпечити спільний період тестових цільових значень. Для варіанта 18 шум додається до тестових входів уже навченої моделі, без зміни цільових значень.

Зміст звіту

Звіт із лабораторної роботи повинен містити:

1. **Титульний аркуш** із назвою роботи, номером варіанта, прізвищем студента та навчальною групою.
2. **Мету роботи.**
3. **Результати реалізації Self-Attention:** тестову послідовність, матрицю ваг, перевірку нормалізації, вихідні представлення й теплову карту.
4. **Результати реалізації Multi-Head Attention:** розмірності тензорів, кількість голів, теплові карти окремих голів та короткий аналіз їхніх відмінностей.
5. **Опис побудованої моделі Transformer:** тип завдання, структура енкодера, основні гіперпараметри, використані шари та функція втрат.
6. **Опис даних і попередньої обробки:** набір даних, поділ на вибірки, токенізація або стандартизація, параметри вхідних послідовностей.
7. **Результати навчання та тестування:** графіки функції втрат і метрик, таблиця тестових результатів, матриця помилок або графіки прогнозування, порівняння з базовою моделлю.
8. **Аналіз ваг уваги навченої моделі:** теплові карти окремих голів, характеристика розподілу уваги та його інтерпретація.

9. **Результати індивідуального завдання:** досліджені конфігурації, таблиця метрик, порівняльні графіки та аналіз впливу параметра.

10. **Програмний код** реалізації механізмів уваги, моделі Transformer, навчання, оцінювання та індивідуального дослідження.

11. **Висновки** щодо якості моделі, особливостей роботи багатоголової уваги, ефективності порівняно з базовою моделлю та впливу параметра, дослідженого за варіантом.

Контрольні запитання

1. Що таке Transformer і в чому полягає його основна відмінність від RNN та LSTM?
2. Що таке механізм Self-Attention і яке завдання він виконує?
3. Яке призначення матриць Query, Key та Value?
4. Як обчислюються оцінки подібності між елементами послідовності?
5. Для чого використовується масштабування на $\sqrt{d_k}$?
6. Яку роль виконує Softmax у механізмі самоуваги?
7. Як формується вихідне представлення елемента за допомогою ваг уваги?
8. Що таке Multi-Head Attention і для чого використовують декілька голів?
9. Як пов’язані розмірність представлення та кількість голів уваги?
10. Для чого в Transformer використовується позиційне кодування?
11. Які основні компоненти містить Transformer Encoder?
12. Яку роль відіграють Feed-Forward Network, залишкові з’єднання та Layer Normalization?
13. Що таке Attention Masking і коли використовують Padding Mask та Causal Mask?
14. Як реалізується багатоголова увага за допомогою tf.keras.layers.MultiHeadAttention?
15. Для чого під час оброблення текстів використовуються токенізація, Embedding та Masked Mean Pooling?
16. Яка функція втрат використовується для багатокласової класифікації текстів?
17. Чим відрізняються метрики Accuracy та F1-score?
18. Як формуються вхідні послідовності для прогнозування часових рядів?
19. Чому під час прогнозування часових рядів необхідно дотримуватися хронологічного поділу даних?
20. Які метрики використовують для оцінювання прогнозування числових показників?
21. Для чого результати Transformer порівнюють із базовими моделями?
22. Як візуалізувати ваги уваги та що можна визначити за тепловими картами?
23. Чому високі ваги уваги не завжди означають визначальний вплив відповідних елементів на результат?

24. Що таке метод Occlusion і як його застосовують для аналізу інтерпретованості Transformer?

25. Як зміна кількості голів, шарів або довжини послідовності впливає на складність і результати моделі?